

Topics covered in the Final of CS 135, Spring 2026

Contents

1	Generate Random Number	1
1.1	Generate a Random Integer in a Range	1
1.2	Generate a Random Floating-Point Number in a Range	2
2	Define and Call Methods	3
2.1	Method Overload	3
2.1.1	Optional: Overload Vs. Override	3
2.2	Pass by Value, Pass by Reference, and Pass by Pointer	3
2.3	Common Mistake: Print Intermediate Result Instead of Return	5
2.4	Common Mistake: Not to Return a Value in Every Possible Situation	5
3	Difference Between == and =	6
4	Array	7
5	Vector	7
6	Pointer	7
6.1	Arrow Operator -> vs. Dot Operator	8
7	Class and Object	13
8	Recursion	13
9	Tips for Taking An Exam	13

1 Generate Random Number

1.1 Generate a Random Integer in a Range

To generate a random integer in `[start, end]`, where `start` $\leq$ `end`, do the following.

1. There are `(end - start + 1)` integers in `[start, end]`.
2. To generate a random integer in `[0, end-start]`, which includes `(end - start + 1)` integers, do the following.

```
int num = rand() % (end - start + 1); //generate a random int in [0 end-start]
```

3. To generate a random integer in `[start, end]`, add `start` to the previous result.

```
int num2 = rand() % (end - start + 1) + start; //generate a random integer in [start, end]
```

- (a) For example, to generate a random integer in `[60, 100]`, do the following:

```
int randVal = rand() % (100 - 60 + 1) + 60; //generate a random integer in [60, 100]
```

1.2 Generate a Random Floating-Point Number in a Range

Generate a random floating-point number in `[start, end]`, where `start ≤ end`.

1. First, generate a random floating-point number in `[0, 1]`.

```
double num = 1.0 * rand() / RAND_MAX;  
//rand() generates a random integer in [0, RAND_MAX].  
//Multiply 1.0 with rand() first, then divide by RAND_MAX to  
//avoid integer division and get a random floating-point number in [0, 1].
```

2. Next, generate a random floating-point number in `[0, end-start]`.

```
double num2 = 1.0 * rand() / RAND_MAX * (end - start);  
//1. rand() generates a random integer in [0, RAND_MAX].  
//2. Multiply 1.0 with rand() first, then divide by RAND_MAX to  
//   avoid integer division and get a random floating-point number in [0, 1].  
//3. Multiply the division result in Step 2 by (end - start) to  
//   get a random floating-point number in [0, end - start].
```

3. Finally, generate a random floating-point number in `[start, end]` by adding `start` to the result in the previous step.

```
double num3 = 1.0 * rand() / RAND_MAX * (end - start) + start;  
//1. rand() generates a random integer in [0, RAND_MAX].  
//2. Multiply 1.0 with rand() first, then divide by RAND_MAX to  
//   avoid integer division and get a random floating-point number in [0, 1].  
//3. Multiply the division result in Step 2 by (end - start) to  
//   get a random floating-point number in [0, end - start].  
//4. Add start to the result of Step 3 to get a random  
//   floating-point number in [start, end].
```

2 Define and Call Methods

2.1 Method Overload

Method overload: same name, similar functionality, different signatures (parameter list is different in number, type, or order). The return type for overloaded methods do not need to be exactly the same.

For example, in Project 3 Task D, there are two methods: `void display(int num)` takes an integer and displays its contents, while `void display(const std::string& str)` takes a string and displays its contents.

```
1 void display(int num);
2 void display(const std::string& str);
```

2.1.1 Optional: Overload Vs. Override

Note that method override is different from method overload. Method override means when a subclass inherits a method from superclass, and if the method in the subclass behaves different from the method in the superclass, then we will override that method in subclass. When method overrides, method name, return type, parameter list all need to be matched to the corresponding method in superclass.

2.2 Pass by Value, Pass by Reference, and Pass by Pointer

Pass by value, pass by reference, pass by pointer (a special case of pass by value)

code link: https://onlinegdb.com/BMKild_y4

```
1 //code link: https://onlinegdb.com/BMKild_y4
2 //Purpose: Demonstrate the differences between
3 //pass-by-value, pass-by-reference,
4 //and pass-by-pointer (also known as pass-by-address).
5 #include <iostream>
6 #include<string>
7
8 //Note that void increase(int num) and void increase2(int& num)
9 //has the same parameter list. For a function that takes
10 //a duplicated copy of passport (pass by value) and
11 //another function that takes an authentic passport (pass by reference),
12 //both of them take a passport as parameter. So their names need
13 //to be different.
14 void increase(int num); //pass by value
15 void increase2(int& num); //pass by reference
16 void increase(int* point);
17
18 int main() {
19     int a = 3;
20     increase(a);
21     std::cout << a << std::endl;
22
23     int b = 3;
```

```

24  increase2(b);
25  std::cout << b << std::endl;
26  //A common mistake is to use
27  //std::cout << increase2(b);
28  //However, the return type of increase2 is void, aka nothing,
29  //you cannot print nothing to the screen.
30
31  int c = 3;
32  increase(&c); //&c is the address of variable c,
33               //an int pointer variable saves the address of an int,
34               //so we are calling function void increase(int *p);
35  std::cout << c << std::endl;
36
37  int *p = new int; // *p is the name of the int variable pointed by p
38  *p = 5;
39  increase(p);
40  std::cout << *p << std::endl;
41
42  delete p; //each new needs to be paired with a delete
43  //(1) since p only points to an int as shown in int *p = new int;
44  //     use delete p.
45  //(2) SUPPOSE p points to a block of ints as in
46  //     int *p = new int[10];
47  //     which applies a dynamically allocated int array with capacity 10,
48  //     and puts the initial address of the array to p.
49  //     That is, p is an int array with dynamically allocated memory.
50  //     Then, when p is no longer needed, use
51  //     delete[] p; to release the memory. CANNOT omit the [] after delete.
52
53  p = nullptr; //avoid dangling pointer problem
54  //(1) When p is set to be nullptr, *p is invalid.
55  //(2) p = nullptr; after delete operation applied to p.
56  //     That is, the address saved in p is released back to the system,
57  //     can no longer use *p, the variable whose address is saved in p.
58
59  //increase(3); //ok
60  //increase2(3); //wrong, since increase2 needs the original copy of a variable,
61  //but 3 is a value of int, not a variable.
62
63  return 0;
64 }
65
66 void increase(int num) {
67     num++;
68 }
69

```

```

70 void increase2(int& num) {
71     num++;
72 }
73
74 void increase(int* point) {
75     (*point)++;
76 }

```

2.3 Common Mistake: Print Intermediate Result Instead of Return

A common mistake is trying to print intermediate result in a method. For example, given an integer, find out whether it is prime or not.

A complete code is in <https://onlinegdb.com/G3V2EEgMp>.

```

1  bool isPrime(int num) {
2      //a prime must be a positive int.
3      //Furthermore, integer 1 is not a prime.
4      //That is, integer <=1 is not a prime.
5      if (num <= 1) {
6          cout << num << " is not prime" << endl;
7          return false;
8      }
9
10     for (int i = 2; i <= sqrt(num); i++) {
11         if (num % i == 0) { //i is a factor of num
12             cout << num << " is not prime" << endl;
13             return false;
14         }
15         else {
16             cout << num << " is prime" << endl;
17             return true;
18         }
19     }
20
21     return true;
22 }

```

Note: when if-, else- or repetition has only one statement, curly parentheses {} is optional but not recommended, however, with more than one statement in its body, is needed; indent is not enough.

2.4 Common Mistake: Not to Return a Value in Every Possible Situation

Another common mistake is not to return a value of the corresponding type in every possible situation. Find errors in the following code segment.

```

1 double getMilesPerGallon() { //numMiles and numGallons are data members.
2     if (numGallons != 0) {
3         return (double)numMiles / numGallons;
4     }
5 }

```

The problem for the above code is, what is the return if numGallons is zero?

3 Difference Between == and =

Pay attention to the difference between == and =. Equality comparison operator == is to compare the left hand side expression equals to the right hand side expression or not, if they are equal, return true, otherwise, return false. Neither side changes its value is not changed. Assignment = operator runs from right to left. First evaluate the right-hand side expression. Second use the value to set the left-side variable. Third the whole assignment expression has the value of the right-hand side expression.

Note that in C/C++, value 0 is false and 1 means true. Look at the following example,
code link: https://onlinegdb.com/4TkVSize_dz

```

1 #include <iostream>
2
3 int main() {
4     //int i = 2.6; //Get a warning when compiled,
5         //cannot put 2.6 to int variable
6     //double d = 2; //work, put an int into a double type
7
8     int i = 2;
9     if (i = 3) //i is changed to be 3 by i = 3.
10         //So expression (i = 3) is 3.
11         //In C++, non-zero number means true.
12         //Run if-body and print i's value is 3.
13         //The condition works in C++ syntax (but not in Java)
14         //but not fit for logic.
15         std::cout << "i's value is " << i << std::endl;
16         //output i's value is 3
17
18     std::string greeting = "hello";
19     std::string str = "hi";
20     if (str == greeting) {
21         std::cout << str << " equals " << greeting << std::endl;
22     }
23     else {
24         std::cout << str << " does not equal " << greeting << std::endl;
25     }
26     //output
27     //hi does not equal hello
28

```

```

29 //if (str = greeting) //not compiled.
30 //a string cannot be a condition.
31 // std::cout << "set " << str << " to be "
32 //      << greeting << std::endl;
33
34 return 0;
35 }

```

4 Array

One-dimensional and two-dimensional array: a collection of same type elements put in adjacent positions

1. Given an array, how to find out the number of elements in this array?
2. How to access an individual element in an array?
3. How to find out sum, average, max, min, and other aggregate operations of array? How to find out whether an element is contained inside an array or not?

5 Vector

1. Vector is like a smart array, with appropriate operations for array provided. You can also think it is an array that can change size. A vector can grow from initial capacity. Know how to define a vector of a given type.
2. Know how to initialize a vector.
3. Use methods `size`, `push_back` and `pop_back` of vector, and access an element of vector using index notation.

6 Pointer

1. Pointer stores the address of a variable. Use pointer for dynamically allocated array.
2. Dereference operator `*` is followed by a pointer. Suppose `p` is a properly declared and initialized int pointer. Then `*p` is the int residing in the address saved in `p`.
3. Arrays and Pointers
 - (a) `int* arr;` is the same as `int arr[];`
 - (b) `(arr+1)` is the address of `arr[1]`. The address of `arr[1]` can also be written as `&arr[1]`.
 - (c) `*(arr+1)` is the same as `arr[1]`. Note that `*(arr + 1)` is not the same as `*arr + 1`, the latter is equivalent to the value of expression `arr[0] + 1`, since `*arr` is `arr[0]`.
4. Apply dynamically allocated memory for a one- or two-dimensional array. Release memory and avoid dangling pointer problem when the array is no longer needed.

6.1 Arrow Operator -> vs. Dot Operator .

1. When we have an object name, use `objectName` followed by dot followed by public data members and methods.

```
1 std::string str("Hello");
2 std::cout << str.length() << std::endl;
```

2. When we have a pointer whose type is a class, use `pointer->` followed by public data members and methods.

There are four ways to initialize a pointer.

- (a) Set a pointer to be `nullptr`. A pointer with `nullptr` value cannot be dereferenced. The following code would result in "Command terminated" exception.

```
1 std::string* pStr = nullptr;
2 std::cout << *pStr << std::endl; //WRONG
```

- (b) Set a pointer to be the address of an existing variable of the same type pointed by the pointer. That is,

```
1 Type variable;
2 Type* ptr = &variable;
3 //Now *ptr is an alias of variable.
```

- i. pointer to an int

```
1 int i; //i is not initialized
2 int* ptr = &i;
3 //*ptr is the same as i.
4
5 *ptr = 5; //after the statement, i is 5.
6 i++; //i changes to 6, and *ptr is changed to 6
7 std::cout << "*ptr = " << *ptr << ", i = " << i << std::endl;
```

- ii. pointer to a string

```
1 //call a default constructor of string to create an instance of empty
   string,
2 //after this statement, str is an empty string.
3 std::string str;
4
5 std::string *pStr = &str; //*pStr is an alias of str,
6 //that is, *pStr and str are the same thing.
7
8 *pStr += "Hello"; //*pStr is changed to be "Hello"
9 //str is "Hello" after the above statement
10
11 std::cout << str << std::endl;
```


A complete code is as follows. See <https://onlinegdb.com/eF5KjiIOn>.

```
1 //Purpose: point to an existing variable of the same type,
2 //that is, set the pointer to the address of that same-type variable.
3 //code link: https://onlinegdb.com/eF5KjiIOn
4 #include <iostream>
5 #include <string>
6
7 int main() {
8     int i; //i is not initialized
9     int* ptr = &i;
10    //*ptr is the same as i.
11
12    *ptr = 5; //after the statement, i is 5.
13    i++; //i changes to 6, and *ptr is changed to 6
14    std::cout << "*ptr = " << *ptr << ", i = " << i << std::endl;
15    //print the following
16    //*ptr = 6, i = 6
17
18    //call a default constructor of string to create an instance of empty
19    string,
20    //after this statement, str is an empty string.
21    std::string str;
22
23    std::string *pStr = &str; //*pStr is an alias of str,
24    //that is, *pStr and str are the same thing.
25
26    *pStr += "Hello"; //*pStr is changed to be "Hello"
27    //str is "Hello" after the above statement
28
29    std::cout << str << std::endl; //print "Hello" without double quotes
30    //a class object can use dot operator to call public methods of that
31    class
32    std::cout << str.length() << std::endl; //print 5
33
34    return 0;
35 }
```

- (c) Apply for a dynamically allocated memory to hold a variable of the same type pointed by the pointer, save the address to the pointer.

Call `delete pointerName`; and `pointerName = nullptr`; when we no longer need to use the variable pointed by `pointerName`.

```
7 //case 1: point to dynamically allocate memory for one element of the same
8 type
9 //That is, use single dynamically allocated string.
10 std::string* singleStrPtr = new std::string("Hello");
11 //*singleStrPtr is a string variable with values "Hello"
```

```

11
12 std::cout << (*singleStrPtr).substr(2, 3) << std::endl; //print "llo"
13 //1. (*singleStrPtr).substr(2, 3) gets a substring of *singleStrPtr,
14 //   starting from index 2, and spreads for 3 characters.
15 //2. When *singleStrPtr is "Hello",
16 //(*singleStrPtr).substr(2, 3) is "llo".
17 //3. () around *singleStrPtr in
18 //   (*singleStrPtr).substr(2, 3) cannot be omitted since
19 //   dot operator . has higher precedence than dereference operator *.
20
21 std::cout << singleStrPtr->substr(2, 3) << std::endl; //print "llo"
22 //(*singleStrPtr).substr(2, 3) can be simplified as singleStrPtr->substr
   (2,3)
23
24 //Suppose we do not need *singleStrPtr now.
25 delete singleStrPtr;
26 singleStrPtr = nullptr;

```

- (d) Apply for dynamically allocated memory to hold an array of the same type pointed by the pointer, save the address to the pointer

The pointer name then can serve as an array name.

Call `delete[] pointerName;` – do not forget `[]` after delete keyword – and `pointerName = nullptr;` when we no longer need to use the variable pointed by `pointerName`.

```

28 //case 2: point to a dynamically allocate memory for an array of elements of
   the same type
29 //That is, use an array of dynamically allocated strings.
30 std::string myStr("Hello"); //can rewrite as
31 //std::string myStr = "Hello";
32
33 int n = myStr.size(); //myStr.size() can be replace with myStr.length()
34 std::string* strArrPtr = new std::string[n]; //strArrPtr is an array of
   strings with n (= 5) elements
35 //strArrPtr can serve as an array name
36
37 for (int i = 0; i < n; i++) {
38     strArrPtr[i] = myStr.substr(i);
39 }
40 //strArrPtr[0] is "Hello"
41 //strArrPtr[1] is "ello"
42 //strArrPtr[2] is "llo"
43 //strArrPtr[3] is "lo"
44 //strArrPtr[4] is "o"
45
46 for (int i = 0; i < n; i++) {
47     std::cout << strArrPtr[i] << std::endl;
48 }

```

```

49 //no longer need the memory pointed by strArrPtr
50 delete[] strArrPtr;
51 strArrPtr = nullptr;
52

```

A complete code can be found at <https://onlinegdb.com/rA8Di6LILw>

Warning: reusing the name `sPtr` for different purposes within the same function, as done in the previous version, is generally considered poor practice.

```

1 //Purpose: illustrate usage of a string pointer and dynamically allocated
  memory
2 //code link: https://onlinegdb.com/rA8Di6LILw
3 #include <iostream>
4 #include <string>
5
6 int main() {
7     //case 1: point to dynamically allocate memory for one element of the same
      type
8     //That is, use single dynamically allocated string.
9     std::string* singleStrPtr = new std::string("Hello");
10    //singleStrPtr is a string variable with values "Hello"
11
12    std::cout << (*singleStrPtr).substr(2, 3) << std::endl; //print "llo"
13    //1. (*singleStrPtr).substr(2, 3) gets a substring of *singleStrPtr,
14    //   starting from index 2, and spreads for 3 characters.
15    //2. When *singleStrPtr is "Hello",
16    //(*singleStrPtr).substr(2, 3) is "llo".
17    //3. () around *singleStrPtr in
18    //   (*singleStrPtr).substr(2, 3) cannot be omitted
19    //   since dot operator . has higher precedence than dereference operator *.
20
21    std::cout << singleStrPtr->substr(2, 3) << std::endl; //print "llo"
22    //(*singleStrPtr).substr(2, 3) can be simplified as singleStrPtr->substr
      (2,3)
23
24    //Suppose we do not need *singleStrPtr now.
25    delete singleStrPtr;
26    singleStrPtr = nullptr;
27
28    //case 2: point to a dynamically allocate memory for an array of elements of
      the same type
29    //That is, use an array of dynamically allocated strings.
30    std::string myStr("Hello"); //can rewrite as
31    //std::string myStr = "Hello";
32
33    int n = myStr.size(); //myStr.size() can be replace with myStr.length()
34    std::string* strArrPtr = new std::string[n]; //strArrPtr is an array of

```

```

    strings with n (= 5) elements
35 //strArrPtr can serve as an array name
36
37 for (int i = 0; i < n; i++) {
38     strArrPtr[i] = myStr.substr(i);
39 }
40 //strArrPtr[0] is "Hello"
41 //strArrPtr[1] is "ello"
42 //strArrPtr[2] is "llo"
43 //strArrPtr[3] is "lo"
44 //strArrPtr[4] is "o"
45
46 for (int i = 0; i < n; i++) {
47     std::cout << strArrPtr[i] << std::endl;
48 }
49
50 //no longer need the memory pointed by strArrPtr
51 delete[] strArrPtr;
52 strArrPtr = nullptr;
53
54 //save the address of an existing variable to a pointer of the same type
55 // --- Use Case 3: Pointer to a stack variable ---
56 // Use a unique name like 'ptrToMyStr'
57 std::string* ptrToMyStr = &myStr;
58 std::cout << "*ptrToMyStr = " << *ptrToMyStr << std::endl;
59 //print "*ptrToMyStr = Hello" without double quotes
60 // No delete needed as it points to stack memory
61
62 //ptrToMyStr does not apply for dynamically allocated memory.
63 //Hence there is no need to release dynamically allocated memory.
64
65 return 0;
66 }

```

3. Warning: if the type is not a class, we cannot use dot operator . For example,

```

int i = 5;
std::cout << i.numDigits() << std::endl;
//i is of int type.
//An int is a primitive type.
//Unlike a class, no operation is associated with a primitive type.
//There is no way to use dot to invoke an nonexistent operation.

```

7 Class and Object

1. Define class (encapsulation) and use its objects. These classes may include arrays as data members.

2. What are class and object?
3. What is encapsulation?
4. Given a class, how to write setters and getters?
5. Given a class, how to write constructors?
6. Is there a return type for constructor? Must constructor share the same name as class?
7. Given constructors of a class, how to create an object from that class?
8. Define a class. Note that data members can be accessed by constructors and methods in the same class and do not need to be passed a parameter.

8 Recursion

Recursion: a function calls itself directly or indirectly. Need to know how to define and/or read recursion codes. Key of recursion is to distinct base case (simplest case) and combine solution to smaller cases to the original problem.

9 Tips for Taking An Exam

1. Be confident. Be calm.
2. If the exam is easy for you, it may be easy for other students. If the exam is difficult for you, it may be difficult for other students as well.
3. Relate the questions with the assignments we have worked.
4. Do not make syntax errors.